

From reaction rates to model lenses

You already know what a derivative means: how rapidly an output changes when an input changes. A **Jacobian** is that familiar idea expanded to many inputs and many outputs.

This guide introduces the notation through concentration changes, then uses it to explain what our model lens measures and why its two derivative estimates disagreed.

The central question

When we nudge the model's internal numbers, are we measuring its local response accurately - or mainly the limits of the measuring method?

Read in three short passes.

Pages 2-8: vectors, matrices and numerical derivatives.

Pages 9-14: the lens and the current evidence.

Pages 15-18: the resolving experiment, glossary and sources.

No prior linear algebra is assumed. Equations are followed by a reading in words. Chemical examples illustrate the mathematics; model coordinates do not literally represent concentrations.

Research snapshot: 9 September 2026. Prepared for Daniel. Small pages, large text and PDF bookmarks for phone reading.

A vector is an ordered list

Suppose a mixture has two concentrations, A and B. We can write both measurements as one object:

$$x = (A, B) = (2, 3) \text{ mM}$$

The first entry is A; the second is B. Both are in millimoles per litre (mM).

That ordered list is a **vector**. Its entries are called **components** or **coordinates**. Here it has two components, so its dimension is two. The order matters.

$$\Delta x = (0.1, -0.2) \text{ mM}$$

Delta means change: add 0.1 mM of A and remove 0.2 mM of B.

Adding the two lists entry by entry gives the new state: **(2.1, 2.8) mM**. Nothing more abstract is needed to understand vector addition.

Transfer to the model

At one token position, a residual vector is a list of internal activation values. In this 4B model it has 2,560 entries. Most entries have no simple human label.

Units matter. Combining concentration and temperature into a list is possible, but measuring that list's overall size requires a declared scaling. We use common units in this example.

A Jacobian is a response table

Consider two hypothetical reaction rates:

$$r_1 = 0.5AB, \qquad r_2 = 0.2B$$

A and B are in mM; rates are in mM/min. The constants are 0.5 mM⁻¹ min⁻¹ and 0.2 min⁻¹.

At A = 2 and B = 3, ask how each rate changes if we vary one concentration while holding the other fixed. These are **partial derivatives**.

$$\frac{\partial r_1}{\partial A} = 0.5B = 1.5$$

For the first rate, an initially small increase in A raises the rate by 1.5 times that concentration change.

Output that changes	Change A	Change B
Rate 1	1.5	1.0
Rate 2	0	0.2

This table is the **Jacobian matrix J**. Every entry has units min⁻¹. Rows name outputs; columns name inputs. The zero says that rate 2 has no direct dependence on A in this toy system.

A matrix is a rectangular table used for a particular operation. Here its operation is to turn small concentration changes into predicted rate changes.

Use each row as a recipe

Use the change list from page 2: $\Delta A = 0.1$ and $\Delta B = -0.2$ mM. Multiply each derivative by its concentration change, then add along the row:

$$\Delta r_1 \approx 1.5(0.1) + 1(-0.2) = -0.05$$

Predicted change in rate 1, in mM/min.

$$\Delta r_2 \approx 0(0.1) + 0.2(-0.2) = -0.04$$

Predicted change in rate 2, in mM/min.

That operation is **matrix-vector multiplication**. Each output gets its own weighted sum. The compact notation is:

$$\Delta y \approx J \Delta x$$

Read: “the change in the output list is approximately J times the change in the input list.”

Recomputing the rates at (2.1, 2.8) gives actual changes of **(-0.06, -0.04)**. The first prediction was slightly off because changing A and B together adds an interaction term:

$$0.5 \Delta A \Delta B = -0.01$$

This is the missing -0.01 mM/min. Making both concentration changes smaller reduces that interaction.

“Linear” means that changes combine by scaling and addition.

“Local linear approximation” means this recipe describes the initial response near the particular state where we took the derivatives.

One entry, one direction, one map

$$J_{ij} = \frac{\partial y_i}{\partial x_j}$$

Entry i,j tells how output i changes when input j changes, holding the other inputs fixed.

The symbol ∂ is the derivative sign used when there are several inputs. An **index** such as i or j is simply a numbered slot in a list.

A **direction** $\mathbf{v}$ specifies a pattern of changes. In two dimensions, $(1, 0)$ changes only the first input. The unit direction $(0.6, 0.8)$ changes both, with overall length one.

$$x \longmapsto x + h \mathbf{v}$$

Start at x and move a distance h in the prescribed direction $\mathbf{v}$. The step h says how far.

$$F(x + h \mathbf{v}) - F(x) \approx h J \mathbf{v}$$

$J \mathbf{v}$ is the initial output response along that direction. Multiplying by h predicts the small finite change.

A column is a special case

Changing just input j selects column j of J . That column lists all the output responses to that single input. Several columns combine when several inputs change.

“Map” means a rule taking one list to another. “Linear map” means that rule is multiplication by a matrix. A model can be nonlinear while its derivative at one state is a linear map.

Measure a slope by two nudges

For one varying input, with all others fixed, we can estimate its derivative by measuring slightly above and below the starting point:

$$D_h F(x) = \frac{F(x+h) - F(x-h)}{2h}$$

Subtract the two output readings, then divide by their input separation, $2h$. This is a central finite difference.

For a Michaelis-Menten-shaped example, divide substrate concentration by K_m and rate by $V_{\max}$. The resulting quantities are dimensionless:

$$f(z) = \frac{z}{1+z}, \quad f'(1) = \frac{1}{4}$$

At $z = 1$, the true local slope is 0.25.

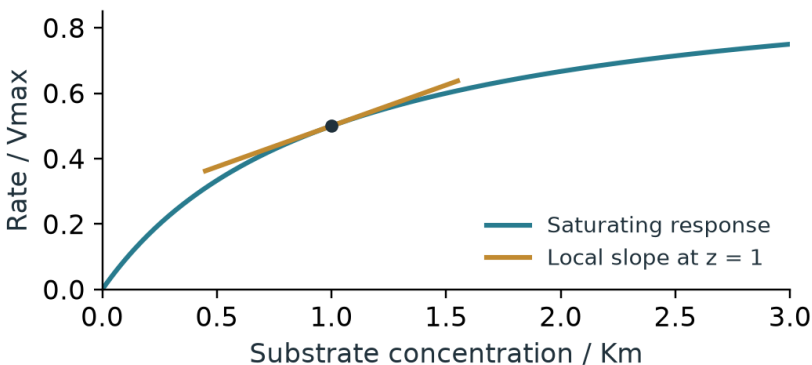


Illustration: a saturating curve and its local slope. This is not model data.

For this curve the finite-difference slope is $1/(4 - h^2)$. With $h = 0.1$ it is 0.250627; with $h = 0.05$ it is 0.250156. Halving h almost quarters the error.

Caution: this saturating curve gives an estimate above the local slope here. Saturation does not automatically imply an estimate that is too small.

A tiny change can disappear

Imagine a readout that displays only two decimal places. With $h = 0.0001$ in the previous example, both measured outputs display **0.50**. Their difference is zero, although the true derivative is 0.25.

Input z	Unrounded output	Display
0.9999	0.499974999	0.50
1.0001	0.500024999	0.50

The unrounded output difference is approximately 0.00005. Divide by the input separation of 0.0002 and the slope is approximately 0.25. After rounding, the same subtraction gives zero.

There are two competing limits: a large step crosses curvature; a tiny step asks the instrument to distinguish readings it cannot resolve.

A familiar laboratory lesson

Reporting more digits after reading an instrument cannot recover a change that its original readout discarded. Converting a rounded tensor to a wider number format has the same limitation.

This decimal display is a teaching example, not a simulation of bfloat16. Binary floating-point gaps vary with magnitude, and rounding can occur repeatedly inside the computation.

Find a useful interval of steps

The finite-difference method needs a step small enough to describe local behavior, yet large enough to produce a resolved change. A useful mathematical picture is:

$$E(h) \lesssim Ah^2 + \frac{B}{h} + C$$

Large-step error: Ah^2 . Amplified arithmetic error: B/h . Additional floor: C .

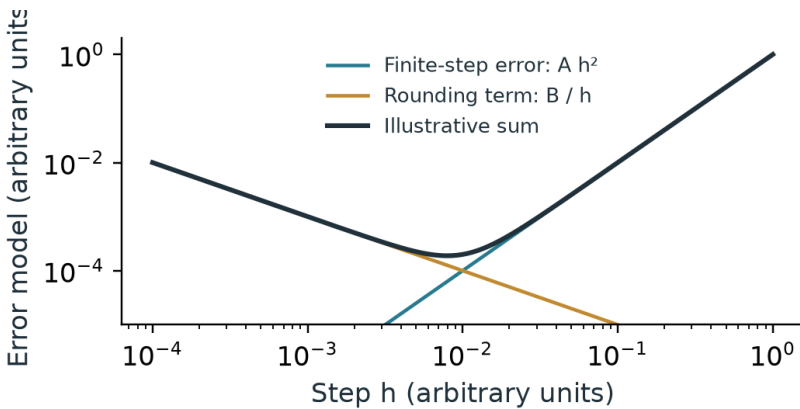


Illustration with $C = 0$, not Gemma data. Logarithmic axes: equal spacing means equal ratios.

A , B and C set the sizes of these effects. The symbol $\lesssim$ means “roughly bounded above by.” One halving ratio cannot identify all three terms. [1]

Smoothness and resolved arithmetic are assumptions. Errors can cancel; a measured curve need not form a perfect V.

What the lens actually maps

A token is a text fragment represented by an integer ID. The model converts it into numbers. As it processes the text, each block updates a vector at each token position: the **residual stream**.

Follow one small perturbation

Source residual → remaining model blocks → final residual → normalization and word-score readout.

Our Jacobian asks how a change at an earlier residual affects the final residual. The current comparison ends **before final normalization**; it does not directly compare derivatives of probabilities or word scores.

An averaged Jacobian lens combines these derivative maps over declared positions and text samples. A **regression lens** instead finds a matrix that predicts later activations from earlier ones on a dataset. These answer different questions.

The final normalization and unembedding translate residuals into token scores, called **logits**. They must be included to make token-level statements. A logit is a score, not itself a probability.

The lens can expose patterns of local sensitivity or prediction under its conventions. It does not identify a thought, prove a feature's causal role at finite amplitude, or make every residual coordinate interpretable.

Read the comparison numbers

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_d^2}$$

A vector norm is an overall length: square its entries, add them, and take the square root. For (3, 4), the answer is 5.

$$\|J\|_F = \sqrt{\sum_{i,j} J_{ij}^2}$$

The Frobenius norm applies the same recipe to every entry of a matrix. It is a size, not a direction.

$$r = \frac{\|\hat{J} - J\|_F}{\|J\|_F}$$

Relative error: the size of the difference between the estimated map and reference map, divided by the reference size.

$$c = \frac{\langle \hat{J}, J \rangle}{\|\hat{J}\|_F \|J\|_F}$$

Cosine: align the two lists of matrix entries. The numerator multiplies matching entries and sums them; the denominator removes their sizes.

Cosine 1 means the same direction; 0 means perpendicular; -1 means opposite. If an estimate is exactly one fifth of the reference, cosine is **1** but relative error is **0.8**.

A hat marks an estimate; a bar marks an average. A low cosine does not prove randomness. Direction is undefined for a zero map. Relative error is also undefined when its reference norm is zero. Near-zero references need an absolute-error check.

What failed - and what did not

Automatic differentiation (AD) used the chain rule. **Finite differences (FD)** subtracted outputs after perturbing activations. Both targeted the same declared map.

Repo layer	Relative error	Cosine
1	1.014	0.015
17	0.973	0.295
33	0.569	0.825

Measured native-bfloat16 results: one held text row, 128 tokens, selected positions 8 and 127. Source: WSD-GOLDEN-4B, c3abdbbe; reproduced in the numerical audit. These are not three independent training runs.

Shallow-layer FD has little directional agreement. Repeating AD gave zero difference: repeatability evidence, not proof of mathematical correctness.

Small-step errors grew sharply, consistent with arithmetic error amplified by division by the step. That does not identify the cause at the original, larger step.

“Exact” is a method label

AD avoids choosing a finite-difference step. Its arithmetic is still finite-precision. It follows the derivative rules of the represented operations, not the literal staircase derivative of machine rounding.

[2]

How big is this “small” step?

The original step was 1% of the overall residual-array norm. That array contained 128 token positions with 2,560 coordinates each. Yet the step was applied to **one coordinate at one position**.

$$\frac{h}{\text{RMS}(H)} = 0.01 \sqrt{128 \cdot 2560} \approx 5.724$$

RMS means root-mean-square: square all entries, average, then take the square root. The step is 5.724 times this global coordinate scale.

That is a reason to test smaller steps. It does not mean every coordinate moved by 5.724 times its own value, nor prove that we left a linear region. Local magnitudes and curvature can differ.

Bfloat16 stores a coarse set of numbers. Just above 1, neighboring representable values include:

$$1.0000000 \longrightarrow 1.0078125$$

The gap is 1/128, about 0.78% of 1. Gaps change with magnitude; this is not a universal absolute tolerance.

Converting a rounded output to float32 afterward cannot recover the discarded information.

Conversely, running whole blocks in float32 changes the arithmetic throughout the path.

Therefore the control must compare float32 AD with float32 FD on the same path. Comparing float32 FD to the old bfloat16 AD result mixes estimator error with a changed computation.

A mean can hide real responses

Suppose two cell populations respond to a perturbation by +2 and -2 reporter units. Their average response is zero:

$$\frac{(+2) + (-2)}{2} = 0$$

Neither population is inactive. The mean has removed their opposing effects.

Our current map averages selected source positions and sums selected target positions. With the two selected positions, it is:

$$\bar{J} = \frac{1}{2}(J_{8 \rightarrow 8} + J_{8 \rightarrow 127} + J_{127 \rightarrow 127})$$

J with p→q means the derivative from source position p to target position q. The 127→8 term vanishes by causality.

The norm of that combined map cannot recover the size of each separate response. A large individual response can disappear in the average.

The saved maps had **no exactly-zero columns**. That means every column retained something after combining responses. It does not tell us how many components or individual responses were lost to rounding.

There are also two directions to a logical claim. If every contribution is zero, the mean is zero. If the mean is zero, the contributions need not be zero. The cell-population example disproves that converse.

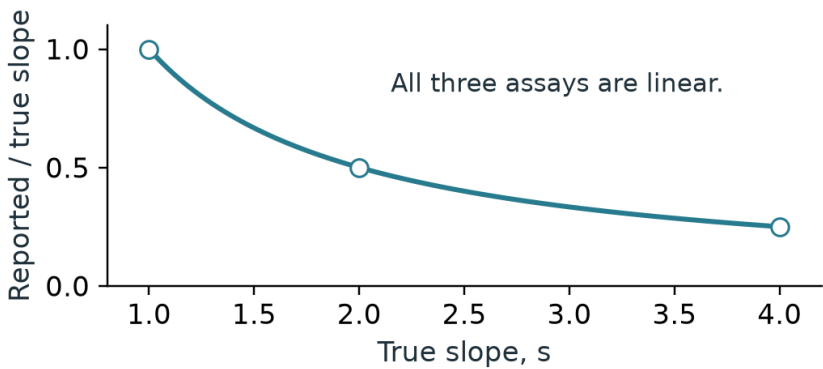
Why correlation does not settle it

The newer saturation argument compared a predicted excursion X with the FD/reference norm ratio Y :

$$X = \frac{hm}{T}, \qquad Y = \frac{f}{a}$$

h is the step; m is median exact-column length; T is a target-size proxy; f and a are FD and exact-map lengths.

At fixed step and target scale, X grows with the true slope; that same slope divides Y . Consider linear assays with slopes 1, 2 and 4, but a faulty estimator reporting 1 for all three:



Synthetic counterexample. A perfect inverse relationship exists even though none of the assays saturates.

The actual cross-layer correlation reproduces at -0.872 . It is compatible with saturation, but cannot identify it. Nonmonotone reversals do not remove the shared-scale problem.

The proposed excursion also uses a map already averaged over positions and a proxy for target size. It is not a measured displacement of one output token. These limitations affect interpretation, not the arithmetic reproduction.

Hold the system still; vary the step

A resolving test needs to change one relevant factor at a time. The recommendation is:

- 1.** Check the intervention itself. Replace a residual with its unchanged capture. The output should remain unchanged under the same forward schedule.
- 2.** Use coherent float32 for both methods. Fix weights, token IDs, positions, directions, attention path and batch schedule. Start with a few directions at batch one.
- 3.** Sweep the step. Save actual perturbations and individual output differences before averaging. Record zero responses, rounding spacing and errors against AD.
- 4.** Compare native bfloat16 separately. If float32 has a useful agreement interval and bfloat16 does not, precision is implicated. If shrinking the step improves float32 greatly, finite-step error is implicated.

Both causes may be present. If both matched sweeps fail, inspect the implementation and the exact quantity being differentiated before averaging more maps.

As of audit 6983d8e, this is a proposed resolving control, not a reported pass. Agreement on a few directions supports those checks; it does not certify every entry of a full matrix.

Local sensitivity and actual edits

The improved measurement design keeps two related outputs:

Local sensitivity

What does the validated derivative predict for an initially small change in these coordinates, at this state?

Finite intervention response

What actually happens when we apply a particular, nonzero edit in the precision used by the running model?

The first can help us locate useful directions and layers. The second tests whether an edit of practical size behaves that way. Their disagreement is information, provided neither is silently substituted for the other.

A sparse dictionary adds another interpretation layer: it tries to express a residual as a combination of feature directions. Applying the lens matrix to one direction is an exact matrix multiplication. Whether that predicts the model's actual response remains an empirical question.

A lens is an instrument with a defined input, output and operating range. Validating that range is what lets a numerical pattern become evidence about the model rather than evidence about our measuring apparatus.

The vocabulary in one place

Activation / residual

Internal numbers at a layer and token position; the residual stream is the state being updated through the blocks.

Vector / coordinate

An ordered list / one numbered entry in that list.

Matrix / Jacobian

A rectangular table acting on vectors / the table of local partial derivatives.

Direction / perturbation

A pattern of relative changes / the actual change applied.

Linear / local

Combining by addition and scaling / describing behavior near one chosen state.

Norm / cosine

Overall size / directional alignment after removing size.

AD / finite difference

Chain-rule differentiation / a slope estimated by subtracting nearby runs.

Precision / resolution

A number format's representational detail / the differences distinguishable at the relevant values and computation.

Aggregation / confound

Combining several readings / an additional changed factor that prevents attributing the result to the factor of interest.

Sources and scope

[1] Choosing finite-difference steps. Shi, Xie, Xuan and Nocedal (2022), *Adaptive finite-difference interval estimation for noisy derivative-free optimization*, SIAM Journal on Scientific Computing. Supports balancing finite-step error against function-evaluation noise; it does not diagnose this transformer.

[Open academic paper](#)

[2] What AD does and does not guarantee. Hückelheim and colleagues (2024), *A taxonomy of automatic differentiation pitfalls*, WIREs Data Mining and Knowledge Discovery, e1555.

[Paper and publication details](#)

Numerical execution details. PyTorch's numerical-accuracy notes explain why mathematically equivalent computations can differ with precision, device and batching.

[Official numerical-accuracy notes](#)

Our evidence. The diagnosis and its control protocol are at 558d424; the saturation audit and reproducible counterexamples are at 6983d8e. Both live in your private repository.

[Diagnosis and resolution protocol](#)

[Saturation audit and numerical evidence](#)

Prepared from committed evidence on 9 September 2026. The chemical systems and error curves are teaching examples. This guide adds no model run and makes no claim that the proposed control has passed.